

Metacharacters

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)

For many types of data, a program also maintains metadata (or meta-information) that it tracks alongside the main data; metadata is simply information that describes or augments the main data. It might include details on how to format data for display, processing instructions, or information on how pieces of the data are stored in memory. There are two basic strategies for representing program data alongside its associated metadata: embedding the metadata in-band or storing the metadata separately, out-of-band.

In-band representation embeds metadata in the data itself. When embedding metadata in textual data, you indicate this information by using special characters called metacharacters or metacharacter sequences. One of the simplest examples of in-band representation is the NUL character terminator in a C string. Out-of-band representation keeps metadata separate from data and associates the two through some external mechanism. String data types in other languages provide a simple example of out-of-band data. Many programming languages (such as C++, Java, PHP, Python, and Pascal) do not have a string terminator character; instead these languages store the string's length in an out-of-band variable.

In many ways, in-band representation is a superior format, as it is often more compact and human readable. However, there are a number of security pitfalls associated with in-band metadata representation that are not a concern for out-of-band metadata. These pitfalls exist because in-band representation creates the potential for overlapping trust domains where explicit boundaries are required. Essentially, in-band metadata representation places both data and metadata within the same trust domain, and parsing routines must handle the logical trust boundaries that exist between data and metadata. However, parsing functions are often very complex, and it can be extremely difficult for developers to account for the security implications of all possible data and metadata combinations.

So far, this chapter has discussed vulnerabilities that can result from mishandling a single in-band metacharacter: the NUL terminator character. However, there are a variety of in-band representations that are common in textual data formats. For example, a slash (/) metacharacter in a filename indicates the beginning or end of a path segment, a dot (.) metacharacter in a hostname indicates a subdomain, and a

space metacharacter in an ASCII-based protocol often denotes the end of an input token. It's not unusual for applications to construct strings by incorporating user-controllable data, as in the following common situations:

- Constructing a filename
- Constructing a registry path (Windows-specific)
- Constructing an e-mail address
- Constructing an SQL statement
- Adding user data to a text file

The following sections examine the potential security ramifications of neglecting to carefully sanitize user input when constructing strings containing metacharacters. Although these sections cover only general situations, later in the chapter you focus on specific examples in contemporary applications, including notorious cases of metacharacter abuse.

Embedded Delimiters

The simplest case of metacharacter vulnerabilities occur when users can embed delimiter characters used to denote the termination of a field. Vulnerabilities of this nature are caused by insufficiently sanitized user input incorporated into a formatted string. For example, say you have a data file containing username and password pairs, with each line in the file in the format `username:password`.

You can deduce that two delimiters are used: the colon (:) character and the newline (\n) character. What if you have the username `bob`, but could specify the password `test\nnewuser:newpassword\n`? The password entry would be written to the file like this:

```
bob:test newuseruser:newpassword
```

You can add an arbitrary new user account, which probably isn't what the developer intended for regular users.

So what would a vulnerable application look like? Essentially, you're looking for a pattern in which the application takes user input that isn't filtered sufficiently and uses it as input to a function that interprets the formatted string. Note that this interpretation might not happen immediately; it might be written to a secondary storage facility and

then interpreted later. An attack of this kind is sometimes referred to a "second-order injection attack."

Note

This phrase "second-order injection attack" has been coined to refer to delayed SQL and cross-site scripting attacks, but it could apply to any sort of stored metacharacter data that's interpreted later.

To see an example of an application that's vulnerable to a basic delimiter injection attack, look at Listing 8-8, which contains the code that writes the password file shown previously.

Listing 8-8. Embedded Delimiter Example

```
use CGI; ... verify session details ... $new_password =  
$query->param('password'); open(IFH, "</opt/passwords.txt") ||  
die("$!"); open(OFH, ">/opt/passwords.txt.tmp") || die("$!");  
while(<IFH>){ ($user, $pass) = split /:/; if($user ne  
$session_username) print OFH "$user:$pass\n"; else  
print OFH "$user:$new_password\n"; } close(IFH); close(OFH);
```

Listing 8-8 does no real sanitization; it simply writes the supplied `password` parameter to the file, so an attacker could add extraneous delimiters.

In general, discovering vulnerabilities of this nature consists of a number of steps:

1. Identify some code that deals with metacharacter strings, including the common examples presented throughout this chapter. Web applications often have a variety of metacharacter strings because they constantly deal with URLs, session data, database queries, and so on. Some of these formats are covered in this chapter; however Web applications are covered in more depth in Chapters 17, "Web Applications," and 18, "Web Technologies."
2. Identify all the delimiter characters that are specially handled. Depending on the situation, different characters take on special meanings. In well-known examples such as format strings and SQL, this chapter specifies the characters you need to be aware of. However, for unique situations, you need to examine the code that interprets the data to find the special characters.
3. Identify any filtering performed on the input, and see what characters or character sequences are filtered (as described in "Input Filters" later in this chapter).

4. Eliminate potentially hazardous delimiter characters from your compiled list that have been filtered out successfully. Any remaining delimiters indicate a vulnerability.

Using this simple procedure, you can quickly evaluate the construction of strings to determine what delimiters or special character sequences could be sneaked into input. The impact of being able to sneak delimiters into the string depends heavily on what the string represents and how it's interpreted. To see this technique in action, look at Listing 8-9, which is a CGI application being launched by a Web server:

Listing 8-9. Multiple Embedded Delimiters

```
BOOL HandleUploadedFile(char *filename) {    unsigned char
buf[MAX_PATH], pathname[MAX_PATH];    char *fname = filename,
*tmp1, *tmp2;    DWORD rc;    HANDLE hFile;    tmp1 =
strrchr(filename, '/');    tmp2 = strrchr(filename, '\\');
if(tmp1 || tmp2)        fname = (tmp1 > tmp2 ? tmp1 : tmp2) +
1;    if(!*fname)        return FALSE;    if(strstr(fname,
".."))        return FALSE;    _snprintf(buf, sizeof(buf),
"\\\\\\?\\%TEMP%\\%s", fname);    rc =
ExpandEnvironmentStrings(buf, pathname, sizeof(pathname));
if(rc == 0 || rc > sizeof(pathname))        return FALSE;
hFile = CreateFile(pathname, ...);    ... read bytes into the
file ... }
```

This code snippet handles an uploaded file from the client and stores the file in a specific temporary directory. Being able to store files outside this directory isn't desirable, of course, but is it safe? Apply the procedure shown previously:

1. Identify some code that deals with format strings. The input string is formatted a couple of ways before it eventually becomes a filename. First, it's added to a statically sized buffer and is prefixed with "\\\\\\?\\%TEMP%\\%s". Second, it's passed to `ExpandEnvironmentStrings()`, where presumably `%TEMP%` is expanded to a temporary directory. Finally, it's used as part of a filename.
2. Identify the set of delimiter characters that are specially handled. Primarily, you want to access a special file or achieve directory traversal, which would involve characters such as `'/'`, `'\\'` and the sequence `".."`. Also, notice that the string is passed to `ExpandEnvironmentStrings()`. Environment variables are denoted with `%`

characters. Interesting!

3. Identify any filtering that's performed. The `strchr()` function is used to find the last slash and then increments past it. Therefore, slashes are out. The code also specifically checks for the double-dot sequence `".."`, so that's out, too.
4. You have eliminated all the usual directory traversal tricks but are left with the `%` character that `ExpandEnvironmentStrings()` interprets. This interpretation allows arbitrary environment variables to be substituted in the pathname. Given that this code is a CGI program, clients could actually supply a number of environment variables, such as `QUERY_STRING`. This environment variable could contain all the sequences that have already been checked for in the original filename. If `"..\..\..\any\pathname\file.txt"` is supplied to `QUERY_STRING`, the client can write to arbitrary locations on the file system.

NUL Character Injection

As you've seen, C uses the NUL metacharacter as a string delimiter, but higher-level languages (such as Java, PHP, and Perl) use counted strings, in which the string contains its length and the NUL character has no special meaning. This difference in interpretation creates situations where the NUL character can be injected to manipulate the behavior of C APIs called by higher level languages. This issue is really just a special case of an embedded delimiter, but it's unique enough that it helps to discuss it separately.

Note

NUL byte injection is an issue regardless of the technology because at some level, the counted string language might eventually interact with the OS. Even a true virtual machine environment, such as Java or .NET, eventually calls base OS functions to do things such as open and close files.

You know that NUL-terminated strings are necessary when calling C routines from the OS and many external APIs. Therefore, a vulnerability may exist when attackers can include NUL characters in a string later handled as a C-style string. For example, say a Perl application opens a file based on some user-provided input. The application requires only text files, so the developer includes a filter requiring that the file end in a `.txt` extension. Figure 8-1 shows an example of a valid filename laid out in memory:

Figure 8-1. C strings in memory

B	O	B	.	T	X	T	\0
---	---	---	---	---	---	---	----

However, what if one of the bytes is a NUL terminator character? After all, Perl doesn't treat the NUL character as a metacharacter. So the resulting string could look like Figure 8-2.

Figure 8-2. C string with NUL-byte injection in memory

B	O	B	\0	.	T	X	T	\0
---	---	---	----	---	---	---	---	----

The function responsible for opening this file would consider the first NUL byte the end of the string, so the .txt extension would disappear and the bob file would be opened.

This scenario is actually quite common in CGI and server-side Web scripting languages. The problems arise when decoding hexadecimal-encoded data (discussed in more depth in "Hexadecimal Decoding" later in this chapter). If the sequence %00 is encountered in input, it's decoded into a single NUL character. If the NUL character isn't handled correctly, attackers can artificially truncate strings while still meeting any other filtering requirements. The following Perl code is a simple example that could generate the altered file name shown Figure 8-2:

```
open(FH, ">$username.txt") || die("$!"); print FH $data; close(FH);
```

The `username` variable in this code isn't checked for NUL characters. Therefore, attackers can NUL terminate the string and create whatever file extensions they choose. The string in Figure 8-2 is just one example, but the NUL character could be used to exploit the server. For example, supplying `execcmd.pl%00` for the `username` will create a file named `execcmd.pl`. A file with the .pl extension can be used to execute arbitrary code on many Web servers.

Most C/C++ programs aren't prone to having NUL bytes injected into user data because they deal with strings as regular C-character arrays. However, there are situations in which unexpected NUL characters can appear in strings. This most commonly occurs when string data is read directly from the network, as shown in Listing 8-10.

Listing 8-10. NUL-Byte Injection with Memory Corruption

```
int read_string(int fd, char *buffer, size_t length) {    int
rc;    char *p;    if(length == 0)        return 1;
length--;    rc = read(fd, buffer, length);    if(rc <= 0)
```

```
return 1;    buffer[length] = '\0';    // trim trailing
whitespace  for(p = &buffer[strlen(buffer)-1]; isspace(*p);
p--)        *p = '\0';    return 0; }
```

The `read_string()` function in Listing 8-10 reads a string and returns it to users after removing trailing whitespace. The developer makes the assumption, however, that the string includes a trailing newline and does not contain any NUL characters (except at the end). If the first byte is a NUL character, the code trims whitespace before the beginning of the buffer, which could result in memory corruption.

The same can be said of dealing with files. When the read primitives are used to read a number of bytes into the buffer from a file, they too might be populated with unexpected NUL characters. This error can lead to problems like the one described previously in Listing 8-10. For example, the `fgets()` function, used for reading strings from files, is designed to read text strings from a file into a buffer. That is, it reads bytes into a file until one of the following happens:

- o

It runs out of space in the destination buffer.

- o

It encounters a newline character (`\n`) or end-of-file (EOF).

So the `fgets()` function doesn't stop reading when it encounters a NUL byte. Because it's specifically intended to deal with strings, it can catch developers unaware sometimes. The following example illustrates how this function might be a problem:

```
if(fgets(buffer, sizeof(buffer), fp) != NULL){
buffer[strlen(buffer)-1] = '\0';    ... }
```

This code is written with the assumption that the trailing newline character must be stripped. However, if the first character is a NUL byte, this code writes another NUL byte before the beginning of the buffer, thus corrupting another variable or program control information.

Truncation

Truncation bugs are one of the most overlooked areas in format string handling, but they can have a variety of interesting results. Developers using memory-unsafe languages can dynamically resize memory at runtime to accommodate for user input or use statically sized buffers based on an expected maximum input length. In statically sized buffers, input that exceeds the length of the buffer must be truncated to fit the buffer size and avoid buffer overflows. Although truncation avoids memory corruption, you might observe interesting side effects from data loss in the shortened

input string. To see how this works, say that a programmer has replaced a call to `sprintf()` with a call to `snprintf()` to avoid buffer overflows, as in Listing 8-11.

Listing 8-11. Data Truncation Vulnerability

```
int update_profile(char *username, char *data) {    char
buf[64];    int fd;    snprintf(buf, sizeof(buf),
"/data/profiles/%s.txt",                username);    fd =
open(buf, O_WRONLY);    ... }
```

The `snprintf()` function (shown in bold) in Listing 8-11 is safe from buffer overflows, but a potentially interesting side effect has been introduced: The filename can be a maximum of only 64 characters. Therefore, if the supplied `username` is close to or exceeds 60 bytes, the buffer is completely filled and the `.txt` extension is never appended. This result is especially interesting in a Web application because attackers could specify a new arbitrary file extension (such as `.php`) and then request the file directly from the Web server. The file would then be interpreted in a totally different manner than intended; for example, specifying a `.php` extension would cause the file to run as a PHP script.

Note

File paths are among the most common examples of truncation vulnerabilities; they can allow an attacker to cut off a mandatory component of the file path (for example, the file extension). The resulting path might avoid a security restriction the developer intended for the code.

Listing 8-12 shows a slightly different example of truncating file paths.

Listing 8-12. Data Truncation Vulnerability 2

```
int read_profile(char *username, char *data) {    char
buf[64];    int fd;    snprintf(buf, sizeof(buf),
"/data/%s_profile.txt",                username);    fd =
open(buf, O_WRONLY);    ... }
```

For Listing 8-12, assume you want to read sensitive files in the `/data/` directory, but they don't end in `_profile.txt`. Even though you can truncate the ending off the filename, you can't view the sensitive file unless the filename is exactly the right number of characters to fill up this buffer, right? The truth is it doesn't matter because you can fill up the buffer with slashes. In filename components, any number of

contiguous slashes are seen as just a single path separator; for example, `////////` and `/` are treated the same. Additionally, you can use the current directory entry `(.)` repetitively to fill up the buffer in a pattern such as this: `././././././.`

Auditing Tip

Code that uses `sprintf()` and equivalents often does so because the developer wants to combine user-controlled data with static string elements. This use may indicate that delimiters can be embedded or some level of truncation can be performed. To spot the possibility of truncation, concentrate on static data following attacker-controllable elements that can be of excessive length.

Another point to consider is the idiosyncrasies of API functions when dealing with data they need to truncate. You have already seen examples of low-level memory-related problems with functions in the `strncpy()` family, but you need to consider how every function behaves when it receives data that isn't going to fit in a destination buffer. Does it just overflow the destination buffer? If it truncates the data, does it correctly NUL-terminate the destination buffer? Does it have a way for the caller to know whether it truncated data? If so, does the caller check for this truncation? You need to address these questions when examining functions that manipulate string data. Some functions don't behave as you'd expect, leading to potentially interesting results. For example, the `GetFullPathName()` function in Windows has the following prototype:

```
DWORD GetFullPathName(LPCTSTR lpFileName, DWORD nBufferLength,  
LPTSTR lpBuffer, LPTSTR *lpFilePart)
```

This function gets the full pathname of `lpFileName` and stores it in `lpBuffer`, which is `nBufferLength TCHARs` long. Then it returns the length of the path it outputs, or 0 on error. What happens if the full pathname is longer than `nBufferLength TCHARs`? The function leaves `lpBuffer` untouched (uninitialized) and returns the number of `TCHARs` required to hold the full pathname. So this failure case is handled in a very unintuitive manner. Listing 8-13 shows a correct calling of this function.

Listing 8-13. Correct Use of `GetFullPathName()`

```
DWORD rc; TCHAR buffer[MAX_PATH], *filepart; DWORD length =  
sizeof(buffer)/sizeof(TCHAR); rc = GetFullPathName(filename,  
length, buffer, &filepart); if(rc == 0 || rc > length) {  
... handle error ... }
```

As you have probably guessed, it's not uncommon for callers to mistakenly just check whether the return value is 0 and neglect to check whether the return code is larger

than the specified length. As a result, if the `lpFileName` parameter is long enough, the call to `GetFullPathName()` doesn't touch the output buffer at all, and the program uses an uninitialized variable as a pathname. Listing 8-14 from the Apache 2.x codebase shows a vulnerable call of `GetFullPathName()`.

Listing 8-14. `GetFullPathName()` Call in Apache 2.2.0

```
apr_status_t filepath_root_case(char **rootpath, char *root,
apr_pool_t *p) { #if APR_HAS_UNICODE_FS
IF_WIN_OS_IS_UNICODE      {      apr_wchar_t *ignored;
apr_wchar_t wpath[APR_PATH_MAX];      apr_status_t rv;
apr_wchar_t wroot[APR_PATH_MAX];      /* ????: This needs
review. Apparently "\\?\d:."      * returns "\\?\d:" as if
that is useful for      * anything.      */      if
(rv = utf8_to_unicode_path(wroot, sizeof(wroot)      /
sizeof(apr_wchar_t), root))      return rv;      if
(!GetFullPathNameW(wroot, sizeof(wpath) /
sizeof(apr_wchar_t), wpath, &ignored))      return
apr_get_os_error();      /* Borrow wroot as a char buffer
(twice as big as      * necessary)      */      if
((rv = unicode_to_utf8_path((char*)wroot,
sizeof(wroot), wpath)))      return rv;
*rootpath = apr_pstrdup(p, (char*)wroot); } #endif      return
APR_SUCCESS; }
```

You can see that the truncation case hasn't been checked for in Listing 8-14. As a result, the `wroot` variable can be used even though `GetFullPathName()` might not have initialized it. You might encounter other functions exhibiting similar behavior, so keep your eyes peeled!

Note

`ExpandEnvironmentStrings()` is one function that behaves similarly to `GetFullPathName()`.

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)



Common Metacharacter Formats

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)

In the previous section, you learned some basic issues with constructing metacharacter strings from user-malleable data. The following sections present specific issues with a number of common metacharacter formats. This is by no means an exhaustive list, but it addresses several of the most common formats. Exploring these formats should reinforce the discussion so far and provide the context needed to identify vulnerability cases in metacharacter formats not covered here.

Path Metacharacters

One of the most common cases of metacharacter vulnerabilities occurs when handling textual representations of path hierarchies. This vulnerability happens most often when programs access files on a file system, but it occurs in other situations too, such as accessing Windows registry keys.

Many systems organize objects into some sort of hierarchy that can be represented textually by constructing a string with each hierarchical path component separated by a delimiter sequence. For file systems, this delimiter is typically a forward slash (/) character in UNIX or a backslash (\) character in Windows. The existence of these delimiter characters in untrusted input might cause vulnerabilities if a program doesn't handle them properly. Exploiting these vulnerabilities could allow an attacker access to objects the developer didn't intend. As a code auditor, you must identify when programs are accessing resources in an unsafe manner that is, when untrusted user input is used to build path components for a resource and when that input can be used to specify objects that shouldn't be accessible. As a quick test, it's a good idea to list resources the application should be able to access, and compare that list with what the application actually allows.

When looking at code dealing with path canonicalization, keep in mind that the truncation issues introduced earlier are particularly relevant, as there's often the opportunity to cut off path elements, such as file extensions for files and subkeys for registry objects.

File Canonicalization

This website uses cookies. Click [here](#) to find out more.

Accept cookies

Applications often receive filenames or paths that are subsequently created or opened for processing. CGI scripts and server-side Web applications, HTTP servers, LPD servers, FTP servers, and privileged local processes are just a few examples of where you see filenames supplied from untrusted sources. Applications that neglect to adequately check untrusted filenames can end up revealing sensitive data to clients, or worse, allowing them to write data to files they shouldn't have access to, which could result in total system compromise.

Each file in a file system has a basic string representation that uniquely identifies its location. This representation typically consists of a device name (optionally), followed by an absolute path, like so:

```
C:\WINDOWS\system32\calc.exe
```

The device is indicated by `c:`, followed by the absolute path where the file resides, `\WINDOWS\system32`, and the filename, `calc.exe`. Although this method is the simplest way to refer to that file, it certainly isn't the only way. In fact, there are many ways to refer to this same file, as shown in these examples:

```
C:\WINDOWS\system32\drivers\..\calc.exe calc.exe .\calc.exe  
..\calc.exe \\?\WINDOWS\system32\calc.exe
```

The process of converting all these different representations into the simplest form is referred to as file canonicalization. When an application requests a file open operation, the kernel resolves the path and verifies permission at each hierarchical level until the destination file is found. However, an application might be at risk when building filenames of data from untrusted sources for example, failing to correctly anticipate how the kernel resolves the requested file path. The subject of file canonicalization is a broad one and differs significantly in Windows and UNIX. For that reason, common issues are addressed in this section, and specifics of UNIX and Windows are covered in Chapters 9, "UNIX I: Privileges and Files," and 11 "Windows I: Objects and the File System."

The most common exploitation of filenames happens if the application neglects to check for directory traversal. In this case, an attacker accesses the parent directory by using the path `..`. When an application builds a pathname that incorporates user-controlled input, it can sometimes be manipulated into unintentionally creating or accessing files outside the subdirectory that file operations should have been restricted to. Applications are vulnerable to these problems when they fail to specifically check for directory traversal or neglect to fully canonicalize the pathname before validating that it's within the expected file system boundaries. Listing 8-15 shows a simple example in Perl.

Listing 8-15. Director

This website uses cookies. Click [here](#) to find out more.

Accept cookies

```
use CGI; ... $username = $query->param('user'); open(FH, "
</users/profiles/$username") || die("$!"); print "<B>User
Details For: $username</B><BR><BR>"; while(<FH>){      print;
print "<BR>" } close(FH);
```

The script in Listing 8-15 attempts to open a user's profile, which is presumably located in the `/users/profiles` directory, but fails to do any sanitization on the `username` variable, which is pulled directly from the query string in the current Web request being serviced. This means attackers could simply specify the user name `../../../../../../etc/passwd` and use this script to print the password file (or any other file of their choosing).

As mentioned, you can diagnose path handling issues by cross-referencing the resources a program requires with the resources it's actually capable of accessing. In Listing 8-15, the resources the program intends to access are user profiles (any files in the `/users/profiles` directory). However, given what you know about file system traversal, you can see that the resources accessible to this program potentially include any files on the system, depending on the user context of the process.

The Windows Registry

Windows operating systems use the Registry as a central repository of system-wide settings that software is free to query and manipulate for its own purposes. Following are the basic Windows registry manipulation functions:

- o

`RegOpenKey()` and `RegOpenKeyEx()` These functions are used for opening a registry key that can subsequently be used in value queries.

- o

`RegQueryValue()` and `RegQueryValueEx()` These functions are used to read data from a specified registry key.

- o

`RegCreateKey()` and `RegCreateKeyEx()` These functions are used to create a new subkey.

- o

`RegDeleteKey()`, `RegDeleteKeyEx()`, and `RegDeleteValue()` The first two functions are used to delete a subkey from the registry. `RegDeleteValue()` leaves the key intact but deletes the value associated with it.

There are a few important considerations in handling registry paths. The first major concern is that tr

This website uses cookies. Click [here](#) to find out more.

Accept cookies

able to cut off part of the key and trick the program into querying the wrong key. The following registry query is vulnerable to truncation:

```
int get_subkey(char *version) {      HKEY hKey;      long rc;      char
buffer[MAX_PATH];      snprintf(buffer, sizeof(buffer),
"\SOFTWARE\MyProduct\%s\subkey2", version);      rc =
RegOpenKeyEx(HKEY_LOCAL_MACHINE, buffer, 0, KEY_READ,
&hKey);      if(rc != ERROR_SUCCESS)          return 1;      ...
RegCloseKey(hKey); }
```

This program reads a configuration parameter by using a version string supplied in data from a remote host. If the version string is long enough, it can fill the buffer and truncate the "subkey2" at the end. Like files, registry keys can have multiple slashes to separate subkey elements, so "\\SOFTWARE\\MyProduct" is equivalent to "\\SOFTWARE\\\\\\\\\\\\\\\\MyProduct" when accessing the key. Furthermore, trailing slashes are truncated as well, so "\\SOFTWARE\\MyProduct" is also equivalent to "\\SOFTWARE\\MyProduct\\\\\\\\\\\\\\\\". Therefore, any time untrusted data is used as part of a registry key, the danger of truncation exists.

Note

The subkey string supplied to `RegOpenKey()` and `RegOpenKeyEx()` can be at most `MAX_PATH` characters long. If the string is any longer, the function returns an error.

As you might have guessed, if attackers can submit additional subkey separators (`\`), they can use them to query arbitrary subkeys or even the base key in the string. The one saving grace is that registry keys are opened in a two-step process: The key must be opened first, and then a particular value is manipulated with another set of functions. However, this two-step process doesn't discount the truncation vulnerability because the attack could still be viable in these situations:

- o

The attacker can manipulate the key name directly.

- o

The attacker wants to manipulate keys, not values.

- o

The application uses a higher-level API that abstracts the key value separation.

- o

The attacker wants to manipulate the default (unnamed) value.

- o

This website uses cookies. Click [here](#) to find out more.

Accept cookies

The value name corresponds to the value the attacker wants to manipulate in another key.

C Format Strings

Format strings are a class of bugs in the `printf()`, `err()`, and `syslog()` families of functions. All these functions convert and print data values of different types to a destination (a string or a filestream). The output data is formatted according to the format string, which contains literal data, and format specifiers that indicate where a data element should be inserted in the stream and what data type it should be converted and displayed as. These functions, therefore, take a variable number of arguments according to how many format specifiers are in the format string. The following code shows an example of calling the `fprintf()` function:

```
if(open(filename, O_RDONLY) < 0){    fprintf(stderr, "[error]: unable
to open filename: %s (%m)\n",      filename);    return(-1); }
```

This code prints a string (the `%s` format specifier) and a system error (the `%m` format specifier).

Note

The `%m` format specifier is an exception to each format specifier having a corresponding argument. It prints a system error string based on the value of the global error indicator `errno`.

Problems happen when untrusted input is used as part or all of the format string argument in any of the functions mentioned previously. Obviously, if users can supply format specifiers that weren't expected, the corresponding arguments don't exist and the values displayed are based on whatever random data happens to be on the program stack. This could allow users to see the program stack or even crash the program by using a format specifier that expects a corresponding pointer argument (such as `%s`, which expects a character pointer to exist on the stack). In addition, one format specifier causes even more problems: `%n`. The `%n` specifier is quite unique in that it doesn't cause any data to be written to the output stream; instead, it takes a corresponding integer pointer argument that gets set to the number of characters output thus far. A legitimate use of `%n` looks like this:

```
int num; printf("test%n", &num);    // sets num to 4
```

The string `test` is printed and the number of output characters is written to `num` (in this case, four). However, this format specifier is quite useful for attackers. The `%n` specifier

This website uses cookies. Click [here](#) to find out more.

Accept cookies

can be exploited to write an arbitrary value to an arbitrary location in memory, which usually results in execution of attacker-controlled code in the current process.

When auditing code, you must ensure that any call to these functions doesn't have a format string derived from untrusted input. You might think a program allowing users to supply the format string isn't likely; after all, why would developers want users to be able to specify format conversions? However, it's happened in a number of applications. One of the most notable examples is in the `SITE EXEC` command of the popular `WU-FTP` daemon. The basic problem is that user-controlled data is passed to the `lreply()` function, as shown:

```
lreply(200, cmd);
```

In this code, the user directly controls the `cmd` variable. Listing 8-16 shows what happens in `lreply()`.

Listing 8-16. Format String Vulnerability in WU-FTPD

```
void lreply(int n, char *fmt,...) {      VA_LOCAL_DECL      if
(!dolreplies) /* prohibited from doing long replies? */
return;      VA_START(fmt);      /* send the reply */
vreply(USE_REPLY_LONG, n, fmt, ap);      VA_END; } void
vreply(long flags, int n, char *fmt, va_list ap) {      char
buf[BUFSIZ];      flags &= USE_REPLY_NOTFMT | USE_REPLY_LONG;
if (n) /* if numeric is 0, don't output one;
use n==0 in place of printf's */      sprintf(buf, "%03d%c", n,
flags & USE_REPLY_LONG ? '-' : ' ');      /* This is somewhat
of a kludge for autospout. I think      * that autospout
should be done differently, but      * that's not my
department. -Kev      */      if (flags & USE_REPLY_NOTFMT)
snprintf(buf + (n ? 4 : 0),      n ? sizeof(buf) - 4 :
sizeof(buf), "%s", fmt);      else      vsnprintf(buf + (n ? 4 :
0),      n ? sizeof(buf) - 4 : sizeof(buf), fmt, ap);
... }
```

As you can see, the second argument to `lreply()` is a format string passed directly to `vreply()`; the `vreply()` function then passes the string as the format specifier to `vsnprintf()`. This example shows how format string vulnerabilities typically occur. They are most likely to happen when a function takes a variable number of arguments and passes that data to an API function for formatting. This type of code occurs most often for logging routines.

Listing 8-17. Format

This website uses cookies. Click [here](#) to find out more.

Accept cookies

```
int log_error(char *fmt, ...) {    char buf[BUFSIZ];
va_list ap;    va_start(ap, fmt);    vsnprintf(buf,
sizeof(buf), fmt, ap);    va_end(ap);    syslog(LOG_NOTICE,
buf); }
```

Listing 8-17 shows a logging routine that format data and pass the result to `syslog()`. However, `syslog()` also performs formatting; so this code is vulnerable to a format string attack.

Auditing Tip

When attempting to locate format string vulnerabilities, search for all instances of `printf()`, `err()`, or `syslog()` functions that accept a nonstatic format string argument, and then trace the format argument backward to see whether any part can be controlled by attackers.

If functions in the application take variable arguments and pass them unchecked to `printf()`, `syslog()`, or `err()` functions, search every instance of their use for nonstatic format string arguments in the same way you would search for `printf()` and so forth.

Because locating format strings is a straightforward process, creating programs that can analyze code (both source and binary) and locate these vulnerabilities automatically isn't too difficult. Many static analysis tools have this capability, including those discussed in Chapter 2. Making use of these tools could be a helpful when verifying whether code is safe from format string attacks.

Shell Metacharacters

Often an application calls an external program to perform a task the external program specializes in, as in the following examples:

- o

A CGI script launches a mail program to send collected form data.

- o

Changing account information on a system might involve launching an editor (`chpass`, for example).

- o

Scheduled execution daemons (`cron` and `at`) call programs scheduled to run as well as a mail program to send results (in some cases).

- o

This website uses cookies. Click [here](#) to find out more.

Accept cookies

Server-side Web applications might call external programs to do some sort of back-end processing.

These examples are only a few possibilities. External application execution happens often and can be prone to security problems. Programs are typically launched in two ways: running the program directly using a function such as `execve()` or `CreateProcess()`, or running it via the command shell with functions such as `system()` or `popen()`. Launching a process via the `exec()` system call replaces the currently running process with the new one or perhaps spawns a new process instance (as with `CreateProcess()`), like so.

```
char *programe = "/usr/bin/sendmail"; char *args[] = { "-s", "hi",
"user@host.com" }; execve(programe, args, envp);
```

In this instance, an application attempts to send an e-mail; however, after calling `execve()`, the current application is replaced with the `sendmail` process. This prevents the original process from writing the e-mail data. To accomplish this, the programmer must fork a new process and set up pipe descriptors. As another option, the program can just run `sendmail` via the shell `popen()` interface. The second option does all the pipe setup and handling internally and is much easier to code. Listing 8-18 shows an example.

Listing 8-18. Shell Metacharacter Injection Vulnerability

```
int send_mail(char *user) {      char buf[1024];      FILE *fp;
snprintf(buf, sizeof(buf),      "/usr/bin/sendmail s
\"hi\" %s", user);      fp = popen(buf, "w");      if(fp ==
NULL)      return 1;      ... write mail ... }
```

When opening commands with this method, any input is subject to interpretation by the shell, so there can be undesirable consequences if certain characters appear in the input stream. To understand these consequences better, return to the following line from Listing 8-18:

```
snprintf(buf, sizeof(buf),      "/usr/bin/sendmail s \"hi\" %s",
user);
```

When `popen()` runs this command, it actually constructs the following command line (assuming the supplied e-mail address is `user@host.com`):

```
/bin/sh c "/usr/bin/sendmail s \"hi\" user@host.com"
```

This website uses cookies. Click [here](#) to find out more.

Accept cookies

The program is being run via the command shell (`sh`), and any shell metacharacters just as if they were typed at the command line or in shell scripts. Returning to the previous example, what if the username is given as `"user@host.com; xterm -- display 1.2.3.4:0"`? The command line that `popen()` constructs now looks like this:

```
/bin/sh c "/usr/bin/sendmail s "hi" user@host.com; xterm display 1.2.3.4:0"
```

The semicolon (`;`) is interpreted as a command separator, so supplying this username doesn't just open `sendmail` as intended; it also sends an `xterm` to a remote host! Parsing shell metacharacters when using `popen()` and similar calls poses an imminent threat to an application when untrusted data is used to build the command line. When auditing applications that use shell capabilities, you need to determine whether arbitrary commands could be run via metacharacter injection. Because the shell has extensive scripting capabilities, quite a number of characters can be useful to attackers. The following list shows the usual suspects:

Dangerous Shell Characters

`;` (separator)

`|` (pipe)

`&` (background)

`<` (redirect)

`>` (redirect)

``` (evaluate)

`!` (not operator)

`-` (argument switch)

`*` (wildcard)

`/` (slash)

`?` (question)

`(` (open parenthesis)

`)` (close parenthesis)

`.` (wildcard)

`" "` (space)

This website uses cookies. Click [here](#) to find out more.

Accept cookies

[ (open bracket)

] (close bracket)

"\t" (tab)

^ (caret)

~ (homedir)

\ (escape)

"\" (backslash)

' (quote)

" (double quote)

"\r" (carriage return)

"\n" (newline)

\$ (variable)

Different shells interpret data differently, so this list isn't complete, but it covers the most common characters. Of course, not all these characters are dangerous in all situations.

You also need to pay close attention to the application being launched. Some applications are inherently dangerous, depending on their function and how they are implemented. Often, you have restrictions on supplying data to these applications; however, the application that's being launched potentially represents a new point of exposure for the caller. (Remember: A chain is only as strong as its weakest link.) Additionally, the called application might have in-band control processing of its own. One notable example is the mail program, as shown in Listing 8-19.

Listing 8-19. An Example of Dangerous Program Use

```
int send_mail(char *name, char *email, char *address) {
 char buf[1024]; FILE *fp; snprintf(buf, sizeof(buf),
 "/usr/bin/mail %s", email); fp = fopen(buf, "w"); if(fp
 == NULL) return 1; fprintf(fp, "Subject: web
 form\n\n"); fprintf(fp, "full name: %s\n", name);
 fprintf(fp, "address: %s\n", address); ... }
```

This website uses cookies. Click [here](#) to find out more.

Accept cookies

For this example, assume the e-mail address has been adequately filtered. So is this program safe? No! The mail program interprets lines beginning with a tilde (~) as a command to run, so if you specify the name or address with a value of "\n~xterm display 1.2.3.4:0", the mail program spawns an xterm.

Obviously, maintaining a detailed knowledge of the inner workings of all programs on a platform or even all applications your team has written can be quite a challenge. Despite this difficulty, when developers decide to call another application, they are crossing a trust boundary and passing control entirely outside the bounds of their applications. Passing control in this way introduces the possibility that the called program could undermine all the calling application's security restrictions. For this reason, it's well worth your time to examine programs instantiated by the application you're auditing, especially if any untrusted input is passed to those programs.

Finally, be mindful of the fact that input doesn't need to be supplied to an external program directly to create vulnerabilities. Attackers might be able to adversely affect an application in a number of other ways, depending on how the program is called and the environment in which it runs. These details tend to be OS specific, however, so they're covered in more depth in Chapters 9 through 12.

Perl open()

The multipurpose capabilities of the Perl `open()` function are worth noting. This function can open both files and processes, so it does the job of the `open()` and `popen()` functions in C. The `open()` function can be called with three arguments (file handle, mode, and filename) or two arguments (file handle and filename). The second method determines in which mode to open the file by interpreting metacharacters that might be at the beginning or end of the filename. These mode characters, listed in Table 8-1, can also direct that the call to the `open()` function should run the data as a command instead of opening a file.

Table 8-1. Mode Character Interpretation in Perl's open() Function

| Byte Sequence | Location  | Meaning                                                                                         |
|---------------|-----------|-------------------------------------------------------------------------------------------------|
| <             | Beginning | Open file for read access.                                                                      |
| >             | Beginning | Open file for write access; create file if it doesn't exist.                                    |
| +<            | Beginning | Open file for read-write access.                                                                |
| +>            | Beginning | Open file for read-write access; create file if it doesn't exist; otherwise, truncate the file. |

This website uses cookies. Click [here](#) to find out more.

Accept cookies

| Byte Sequence | Location  | Meaning                                                                                          |
|---------------|-----------|--------------------------------------------------------------------------------------------------|
| >>            | Beginning | Open file for write access but don't truncate; append to the end of the file.                    |
| +>>           | Beginning | Open file for read-write access but don't truncate; append to the end of the file.               |
|               | Beginning | This argument is a command, not a filename. Create a pipe to run this command with write access. |
|               | End       | This argument is a command, not a filename. Create a pipe to run this command with read access.  |

When no mode characters are specified, the file is opened for just read access, just as if the file argument contains a leading <. This programming practice is a dangerous, however, because if attackers can specify the filename (or at least the filename's leading component), they can choose the mode in which the file is opened! Here's an example of a dangerous call:

```
open(FH, "$username.txt") || die("$!");
```

The second argument contains no leading mode characters, allowing users to specify arbitrary mode characters. The most dangerous is the pipe character, which causes an arbitrary command to run. For example, by specifying the username as "| xterm d 1.2.3.4:0;", users can spawn a remote xterm with this script! The same applies if the last part of the filename can be specified, as in this example:

```
open(FH, "/data/profiles/$username");
```

In this case, remote execution could be achieved by specifying a username such as "blah; xterm -d 1.2.3.4:0 |". If users can't control the beginning or the end of a filename, they can't insert pipes for running commands.

#### Note

You might think that if attackers controlled a little data in the middle of the file argument, they could achieve remote execution by specifying an argument such as "blah; xterm d 1.2.3.4:0|%00", using the NUL-byte injection technique. Although this technique chops (|) as the last character, it can still be used to interpret the pipe

This website uses cookies. Click [here](#) to find out more.

Accept cookies

Also, keep in mind that the `+>` mode opens a file for read-write access and truncates the file to 0 bytes. This mode is somewhat unique because the file can be modified. Say untrusted data is supplied as the `username` variable in the following call:

```
open(FH, "+>$username.txt");
```

If the `username` variable begins with a `>`, the file is opened in append mode and isn't truncated. Depending on the application's specifics, this result might have interesting implications, such as reading data created by a previous user session.

Apart from this special case, if a mode is specified for opening the file, the call is safe, right? No, there's more! The `open()` function in Perl also duplicates file descriptors for you. If the mode argument is followed by an ampersand (`&`) and the name of a known file handle (`STDIN`, `STDOUT`, `STDERR`), `open()` duplicates the file descriptor with a matching mode (such as a leading `<` for `STDOUT`). Additionally, you can specify any file descriptor number you want with the syntax `&=<fd number>`. Take a look at this example:

```
open(ADMIN, "+>>/data/admin/admin.conf"); ... open(USER, ">$userprofile");
```

This code fragment assumes that the `ADMIN` file hasn't been closed when the second call to `open()` is made. It enables attackers to write to the `/data/admin/admin.conf` file. They simply need to know the correct file descriptor number and supply it as the `userprofile` value, such as `&=3` if `admin.conf` is opened as file descriptor 3. Note that the `open()` call might be exploitable in the following example too:

```
open(ADMIN, "+>>/data/admin/admin.conf"); ... open(USER, ">$userprofile.txt");
```

If attackers can insert NUL bytes, they can supply the `userprofile` value `&=3`, and the file descriptor is duplicated correctly. However, the three-argument version of `open()` would render this code no longer vulnerable because it requires explicitly setting the mode in the second parameter. So you can consider three-argument calls to `open()` to be secure, for the most part.

### Auditing Tip

You might find a vulnerability in which you can duplicate a file descriptor. If you have access to an environment similar to one in which the script is running, use `lsof` or a similar tool to determine what file descriptors are open when the process runs. This tool should help you see what you might have access to.

This website uses cookies. Click [here](#) to find out more.

Accept cookies

## SQL Queries

SQL is a standard language for interacting with a relational database management system (RDBMS). You most likely encounter SQL in the context of Web applications, when examining server-side scripts or applications that process input from browsers and issue queries to a back-end database. Incorrect handling of input to these queries can result in severe vulnerabilities. This discussion focuses on dynamically constructed queries in normal SQL syntax. Chapter 17, "Web Applications," expands this coverage to address parameterized queries and stored procedures.

The most common SQL-related vulnerability is SQL injection. It occurs when input is taken from request data (post variables, forms, or cookies) and concatenated into a query string issued against the database. Listing 8-20 is a simple example in PHP and MySQL.

Listing 8-20. SQL Injection Vulnerability

```
$username = $HTTP_POST_VARS['username']; $password =
$HTTP_POST_VARS['passwd']; $query = "SELECT * FROM logintable
WHERE user = '" . $username . "' AND pass = '" .
$password . "'"; ... $result = mysql_query($query);
if(!$result) die_bad_login(); ...
```

This query is vulnerable to SQL injection because users can supply unfiltered input for the `passwd` and `username` variables. Attackers could easily submit a string such as `"bob' OR pass <> 'bob"` for both parameters, which results in the following query being issued against the database:

```
SELECT * from logintable WHERE user = 'bob' OR user <> 'bob' AND
pass = 'bob' OR pass <> 'bob'
```

In this example, attackers take advantage of the script not filtering the single-quote character (`'`), which allows them to supply the closing quote and include their own SQL statements. Of course, a single quote isn't the only way to manipulate an SQL query. Dealing with unbounded integer fields (or any data not enclosed in quotes) might cause problems, too. Developers don't expect these fields to contain nonnumeric data, so they often don't check for other data types, particularly if the data is taken from a hidden field or cookie. Take a look at this example:

```
$order_id = $HTTP_POST_VARS ['hid_order_id']; $query = "SELECT * FROM
orders WHERE docText">This example is similar to the previous one,
except the ord This website uses cookies. Click here to find out more. that should
contain an int Accept cookies nised by
```

supplying a value such as "1 OR 1=1" for `hid_order_id`. In this case, you could expect the application to return all orders in the system.

#### Note

PHP and MySQL provide mechanisms for cleaning strings to help mitigate the risk of this attack. Some examples of filtering functions are `mysql_real_escape_string()`, `dbx_escape_string()`, and `pg_escape_string()`. However, filtering isn't the most reliable method of addressing this issue, as it is still in-band representation and could be vulnerable to unforeseen attack vectors and errors in the filtering routines. Chapter 17 discusses parameterized queries as an out-of-band query method that provides a more effective method of protection.

The impact of SQL injection vulnerabilities varies depending on the RDBMS and database structure. Some databases limit injected queries to the scope of the initial statement; for instance, a vulnerability in an Oracle `SELECT` statement allows injecting only additional `SELECTS` or function calls. On the other end of the spectrum, some databases allow the injected statement almost unlimited functionality; Microsoft SQL Server allows terminating the current statement with a semicolon, and then appending another statement.

In many cases, the database contents are attackers' final goal, so they are happy with any vulnerability that grants arbitrary database interaction. However, the attack could also be extended to the hosting system. Many RDBMS implementations have stored procedures for running system commands or arbitrary processes. Attackers can use these procedures unless the database session is explicitly denied access. Failing that approach, the RDBMS implementation itself might expose vulnerabilities. Many stored procedures and functions are implemented in other languages, which can expose a host of other potential vulnerabilities (discussed more in Chapter 17).

You might also see a cookie notice banner. This website uses cookies. Click [here](#) to find out more. You can also [Accept cookies](#) or [Decline cookies](#).

languages using fixed-size buffers; attackers can fill a buffer enough to eliminate trailing clauses. Of course, most developers prefer to use languages with counted string classes when handling SQL queries. Still, it's worth keeping this attack in mind if you encounter C/C++ front ends that manipulate SQL. Listing 8-21 shows an example.

Listing 8-21. SQL Truncation Vulnerability

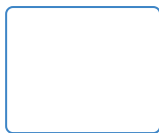
```
int search_orders(char *post_detail, char *sess_account) {
char buf[1024]; int rc; post_detail =
escape_sql(post_detail); sess_account =
escape_sql(sess_account); snprintf(buf, sizeof(buf),
"SELECT * FROM orders WHERE detail LIKE " \
"\'%%%s%\' AND account = \'%s\'", post_detail,
sess_account); rc = perform_query(buffer);
free(post_detail); free(sess_account); if(rc > 0)
return 1; return 0; }
```

Assume that the `search_orders()` function in Listing 8-21 allows users to search through their own orders, but no one else's. The `escape_sql()` function prevents users from injecting any control characters, and the `sess_account` variable is provided from the session. This means users can manipulate only the length and content of the `post_detail` variable. However, they can pad `post_detail` with enough percent signs (%) that it causes `snprintf()` to cut off the `AND` clause that restricts the query to current users. The resulting query then retrieves all orders, regardless of the user.

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)

This website uses cookies. Click [here](#) to find out more.

Accept cookies



# Metacharacter Filtering

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)

The potential issues associated with metacharacters often necessitates a more defensive coding strategy. Generally, this strategy involves attempting to detect potential attacks or sanitize input before it's interpreted. There are three basic options:

- Detect erroneous input and reject what appears to be an attack.
- Detect and strip dangerous characters.
- Detect and encode dangerous characters with a metacharacter escape sequence.

Each of these options has its uses, and each opens the potential for new vulnerabilities. The first two options attempt to eliminate metacharacters outright, so they share certain commonalities addressed in the next section. The third option involves a number of unique concerns, so it is addressed separately in "Escaping Metacharacters."

## Eliminating Metacharacters

Rejecting illegal requests and stripping dangerous characters are similar strategies; they both involve running user data through some sort of sanitization routine, often using a regular expression. If the disallowed input is rejected, any request containing illegal metacharacters is simply discarded. This approach usually includes some sort of error indicating why the input wasn't allowed, as shown in this example:

```
if($input_data =~ /[^\A-Za-z0-9_]/){ print "Error! Input data
contains illegal characters!"; exit; }
```

In this example, the `input_data` variable is checked for any character that isn't alphanumeric, an underscore, or a space. If any of these characters are found, an error is signaled and processing terminates.

This website uses cookies. Click [here](#) to find out more.

Accept cookies

With character stripping, the input is modified to get rid of any violations to the restrictions, and then processing continues as normal. Here's a simple modification of the previous example:

```
$input_data =~ s/[^A-Za-z0-9]/g;
```

Each option has its strengths and weaknesses. Rejection of dangerous input lessens the chance of a breach because fewer things can go wrong in handling. However, a high false-positive rate on certain inputs might cause the application to be particularly unfriendly. Stripping data elements is more dangerous because developers could make small errors in implementing filters that fix up the input stream. However, stripping input may be considered more robust because the application can handle a wide variety of input without constantly generating errors.

Both approaches must account for how strong their filter implementation is; if they don't catch all the dangerous input, nothing that happens afterward matters much! There are two main types of filters: explicit deny filters (black lists) and explicit allow filters (white lists). With an explicit deny filter, all data is assumed to be legal except the specific characters deemed dangerous. Listing 8-22 is an example of an explicit deny filter implementation.

Listing 8-22. Character Black-List Filter

```
int islegal(char *input) { char *bad_characters = "\"\\|<>&-*";
for(; *input; input++){
if(strchr(bad_characters, *input) return 0; }
return 1; }
```

As you can see, this filter allows any characters except those in the `bad_characters` set. Conversely, an explicit allow filter checks for characters known to be legal, and anything else is assumed illegal, as shown in Listing 8-23.

Listing 8-23. Character White-List Filter

```
int islegal(char *input) { for(; *input; input++){
if(!isalnum(*input) && *input != '_' && !isspace(*input))
return 0; } return 1; }
```

This example is similar to Listing 8-22, except it's testing for the existence of each character in a set of legal characters.

White-list filters are generally considered

more secure. When the accept set is large, however, using an explicit deny filter might be more appropriate.

When reviewing code containing filters of either kind, you must determine whether the application has failed to account for any dangerous input. To do this, you should take these steps:

1. Make a list of every input the filter allows.
2. Make a list of every input that's dangerous if left in the input stream.
3. Check whether there are any results from the intersection of these two lists.

Step 1 is straightforward and can be done from just reading the code; however, step 2 might require more creativity. The more knowledge you have about the component or program interpreting the data, the more thorough analysis you can perform. It follows, therefore, that a good code auditor should be familiar with whatever data formats they encounter in an assessment. For example, shell programming and SQL are metadata formats commonly used in web applications.

#### Insufficient Filtering

When you already have a thorough knowledge of the formats you deal with, there's usually the temptation to not make allowed input lists. You might instead choose to draw on your existing knowledge to assess the filter's strength. This approach may be adequate, but it also increases your chances of missing subtle vulnerabilities, just as the application developer might. For example, take a look at Listing 8-24, which demonstrates a filtering vulnerability in the PCNFSD server.

Listing 8-24. Metacharacter Vulnerability in PCNFSD

```
int suspicious (s) char *s; { if(strpbrk(s, ";|&<>`'#!?*(
[]^") != NULL) return 1; return 0; }
```

A filter is constructed to strip out dangerous characters before the data is passed to `popen()`. The developers have a fairly complete reject set, but they missed a character. Can you see what it is? That's right: it's the newline (`'\n'`) character. If a newline character is inserted in the input stream, the shell treats the data before it as one command and the data after it as a new command, thus allowing attackers to run arbitrary commands. This example is interesting because the newline character is often forgotten when constructing filters. [This website uses cookies. Click here to find out more.](#) [Link about other](#) [Accept cookies](#) [but the newline](#)

character, demonstrating that even experienced programmers can be familiar with a system yet make oversights that result in vulnerabilities.

Even when you're familiar with a format, you need to keep in mind the different implementations or versions of a program. Unique extensions might introduce the potential for variations of standard attacks, and data might be interpreted more than once with different rules. For example, when sanitizing input for a call to `popen()`, you need to be aware that any data passed to the program being called is interpreted by the command shell, and then interpreted again differently by the program that's running.

#### Character Stripping Vulnerabilities

There are additional risks when stripping illegal characters instead of just rejecting the request. The reason is that there are more opportunities for developers to make mistakes. In addition to missing potentially dangerous characters, they might make mistakes in implementing sanitization routines. Sometimes implementations are required to filter out multicharacter sequences; for example, consider a CGI script that opens a file in a server-side data directory. The developers want to allow users to open any file in this directory, and maybe even data in subdirectories below that directory. Therefore, both dot (.) and slash (/) are valid characters. They certainly don't want to allow user-supplied filenames outside the data directory, such as `../../../../etc/passwd`; so the developers strip out occurrences of the `../` sequence. An implementation for this filter is shown in Listing 8-25.

Listing 8-25. Vulnerability in Filtering a Character Sequence

```
char *clean_path(char *input) { char *src, *dst; for(src
= dst = input; *src;){ if(src[0] == '.' && src[1] ==
'.' && src[2] == '/'){ src += 3;
memmove(dst, src, strlen(src)+1); continue;
} else *dst++ = *src++; } *dst = '\0';
return input; }
```

Unfortunately, this filtering algorithm has a severe flaw. When a `../` is encountered, it's removed from the stream by copying over the `../` with the rest of the path. However, the `src` pointer is incremented by three bytes, so it doesn't process the three bytes immediately following a `../` sequence! Therefore, all an attacker needs to do to bypass it is put one double dot exactly after another, because the second one is missed. For example, input such as `../../../../test.txt` is converted to `../test.txt`. Listing 8-26 shows how to fix the incorrect filter.

Listing 8-26. Vulnera

This website uses cookies. Click [here](#) to find out more.

Accept cookies

```
char *clean_path(char *input) { char *src, *dst;
for(src = dst = input; *src;){ if(src[0] == '.' &&
src[1] == '.' && src[2] == '/'){ memmove(dst,
src+3, strlen(src+3)+1); continue; } else
*dst++ = *src++; } *dst = '\0'; return input; }
```

Now the algorithm removes ../ sequences, but do you see that there's still a problem? What happens if you supply a file argument such as ....//hi? Table 8-2 steps through the algorithm.

Table 8-2. Desk-Check of clean\_path with Input ....//hi

| Iteration | Input    | Output                  |
|-----------|----------|-------------------------|
| 1         | ....//hi | .                       |
| 2         | ...//hi  | ..                      |
| 3         | ..//hi   | .. (Nothing is written) |
| 4         | /hi      | ../                     |
| 5         | hi       | ../h                    |
| 6         | i        | ../hi                   |

This algorithm demonstrates a subtle issue common to many multicharacter filters that strip invalid input. By supplying characters around forbidden patterns that combine to make the forbidden pattern, you have the filter itself construct the malicious input by stripping out the bytes in between.

Auditing Tip

When auditing multicharacter filters, attempt to determine whether building illegal sequences by constructing embedded illegal patterns is possible, as in Listing 8-26.

Also, note that these attacks are possible when developers use a single substitution pattern with regular expressions, such as this example:

```
$path =~ s/\.\\.\\.\\/g;
```

This approach is prevalent in several programming languages (notably Perl and PHP).

This website uses cookies. Click [here](#) to find out more.

Accept cookies

## Escaping Metacharacters

Escaping dangerous characters differs from other filtering methods because it's essentially nondestructive. That is, it doesn't deny or remove metacharacters but handles them in a safer form. Escaping methods differ among data formats, but the most common method is to prepend an escape metacharacter (usually a backslash) to any potentially dangerous metacharacters. This method allows these characters to be safely interpreted as a two-character escape sequence, so the application won't interpret the metacharacter directly.

When reviewing these implementations, you need to be mindful of the escape character. If this character isn't treated carefully, it could be used to undermine the rest of the character filter. For example, the following filter is designed to escape the quote characters from a MySQL query using the backslash as an escape metacharacter:

```
$username =~ s/\"'\"*\\"$1/g; $passwd =~ s/\"'\"*\\"$1/g; ... $query
= "SELECT * FROM users WHERE user='" . $username . "' AND pass = '"
. $passwd . "'";
```

This query replaces dangerous quote characters with an escaped version of the character. For example, a username of "bob' OR user <> 'bob" would be replaced with "bob\' OR user <> \'bob". Therefore, attackers couldn't break out of the single quotes and compromise the application. The regular expression pattern neglects to escape the backslash character (\), however, so attackers still have an avenue of attack by submitting the following:

```
username = bob\' OR username = passwd = OR 1=1
```

This input would create the following query after being filtered:

```
SELECT * FROM users WHERE user='bob\\' OR username = ' ' AND pass = '
OR 1=1
```

The MySQL server interprets the double-backslash sequence after bob as an escaped backslash. This prevents the inserted backslash from escaping the single quote, allowing an attacker to alter the query.

### Note

Escape characters vary between SQL implementations. Generally, the database supports the slash-quote (\') or double-apostrophe (``) escape sequences. However, developers might confuse which escape sequence is supported and accidentally use the wrong sequence.

This website uses cookies. Click [here](#) to find out more.

Accept cookies

## Metacharacter Evasion

One of the most interesting security ramifications of escaping metacharacters is that the encoded characters can be used to avoid other filtering mechanisms. As a code auditor, you must determine when data can be encoded in a manner that undermines application security. To do this, you must couple decoding phases with relevant security decisions and resulting actions in the code. The following steps are a basic procedure:

1. Identify each location in the code where escaped input is decoded.
2. Identify associated security decisions based on that input.
3. If decoding occurs after the decision is made, you have a problem.

To perform this procedure correctly, you need to correlate what data is relevant to the action performed after the security check. There's no hard and fast method of tying a decoding phase to a security decision, but one thing you need to consider is that the more times data is modified, the more opportunities exist for fooling security logic. Beyond that, it's just a matter of understanding the code involved in data processing. To help build this understanding, the following sections provide specific examples of how data encodings are used to evade filters.

### Hexadecimal Encoding

HTTP is discussed in more detail in later chapters; however, this discussion of encoding would be remiss if it didn't address the standard encoding form for URIs and query data. For the most part, all alphanumeric characters are transmitted directly via HTTP, and all other characters (excluding control characters) are escaped by using a three-character encoding scheme. This scheme uses a percent character (%) followed by two hexadecimal digits representing the byte value. For example, a space character (which has a hexadecimal of 0x20) uses this three-character sequence: %20.

HTTP transactions can also include Unicode characters. Details of Unicode are covered in "Character Sets and Unicode" later in this chapter, but for this discussion, you just need to remember that Unicode characters can be represented as sequences of one or two bytes. For one-byte sequences, HTTP uses the hexadecimal encoding method already discussed. However, for two-byte sequences, Unicode characters can be encoded with a six-character sequence consisting of the string %u or %U followed by four hexadecimal digits. These digits represent the 16-bit value of a Unicode character. These alternate

This website uses cookies. Click [here](#) to find out more.

Accept cookies

through character filters. To understand the problem, look at the sample code in Listing 8-27.

Listing 8-27. Hex-Encoded Pathname Vulnerability

```
int open_profile(char *username) { if(strchr(username,
'/')) { log("possible attack, slashes in username");
return 1; } chdir("/data/profiles"); return
open(hexdecode(username), O_RDONLY); }
```

This admittedly contrived example has a glaring security problem: the `username` variable is checked for slashes (/) before hexadecimal characters are decoded. Using the coupling technique described earlier, you can associate decoding phases, security decisions, and actions as shown in this list:

- Decision If `username` contains a / character, it's dangerous (refer to line 3 in Listing 8-27).
- Decoding Hexadecimal decoding is performed on input after the decision (refer to line 10).
- Action `Username` is used to open a file (refer to line 10).

So a `username` such as `..%2F..%2Fetc%2Fpasswd` results in this program opening the system password file. Usually, these types of vulnerabilities aren't as obvious. Decoding issues are more likely to occur when a program is compartmentalized, and individual modules are isolated from the decoding process. Therefore, the developer using a decoding module generally isn't aware of what's occurring.

#### Note

Hexadecimal encoding is also a popular method for evading security software (such as IDSs) used to detect attacks against Web servers. If an IDS fails to decode hexadecimal encoded requests or decodes them improperly, an attack can be staged without generating an alert.

Handling embedded hexadecimal sequences is usually simple. A decoder can generally do two things wrong:

- Skip a NUL byte.

This website uses cookies. Click [here](#) to find out more.

Accept cookies

o

Decode illegal characters.

Earlier in this chapter, you examined a faulty implementation that failed to check for NUL bytes (see Listing 8-5). So this coverage will concentrate on the second error, decoding illegal characters. This error can happen when assumptions are made about the data following a % sign. Two hexadecimal digits are expected follow a % sign. Listing 8-28 shows a typical implementation for converting those values into data.

Listing 8-28. Decoding Incorrect Byte Values

```
int convert_byte(char byte) { if(byte >= 'A' && byte <=
'F') return (byte - 'A') + 10; else if(byte >= 'a'
&& byte <= 'f') return (byte - 'a') + 10; else
return (byte - '0'); } int convert_hex(char *string) { int
val1, val2; val1 = convert_byte(string[0]); val2 =
convert_byte(string[1]); return (val1 << 4) | val2; }
```

The `convert_byte()` function is flawed, in that it assumes the byte is a number character if it's not explicitly a hexadecimal letter (as shown in the bolded lines). Therefore, invalid hex characters passed to this function (including the characters A through F) produce unexpected decoded bytes. The security implication of this incorrect decoding is simple; any filters processing the data in an earlier stage miss values that can appear in the resulting output stream.

#### HTML and XML Encoding

HTML and XML documents can contain encoded data in the form of entities, which are used to encode HTML rendering metacharacters. Entities are constructed by using the ampersand sign (&), followed by the entity abbreviation, and terminated with a semicolon. For example, to represent an ampersand, the abbreviation is "amp," so `&amp;` is the encoded HTML entity. A complete list of entities is available from the World Wide Web Consortium (W3C) site at [www.w3c.org](http://www.w3c.org).

Even more interesting, characters can also be encoded as their numeric codepoints in both decimal and hexadecimal. To represent a codepoint in decimal, the codepoint value is prepended with `&#`. For example, a space character has the decimal value 32, so it's represented as `&#32`. Hex encoding is similar, except the value is prepended with `&#x`, so the space character (0x20) is represented as `&#x20`. Two-byte Unicode characters can also be specified with five decimal or four hexadecimal digit sequences. This encoding form is susceptible to the same basic vulnerabilities that hexadecimal decoders might have, such as embedding NUL characters, evading filters, and assuming the

This website uses cookies. Click [here](#) to find out more.

Accept cookies

## Note

Keep in mind that HTML decoding is normally handled by a client browser application. However, using this encoding form in XML data does open the possibility of a variety of server-directed attacks.

## Multiple Encoding Layers

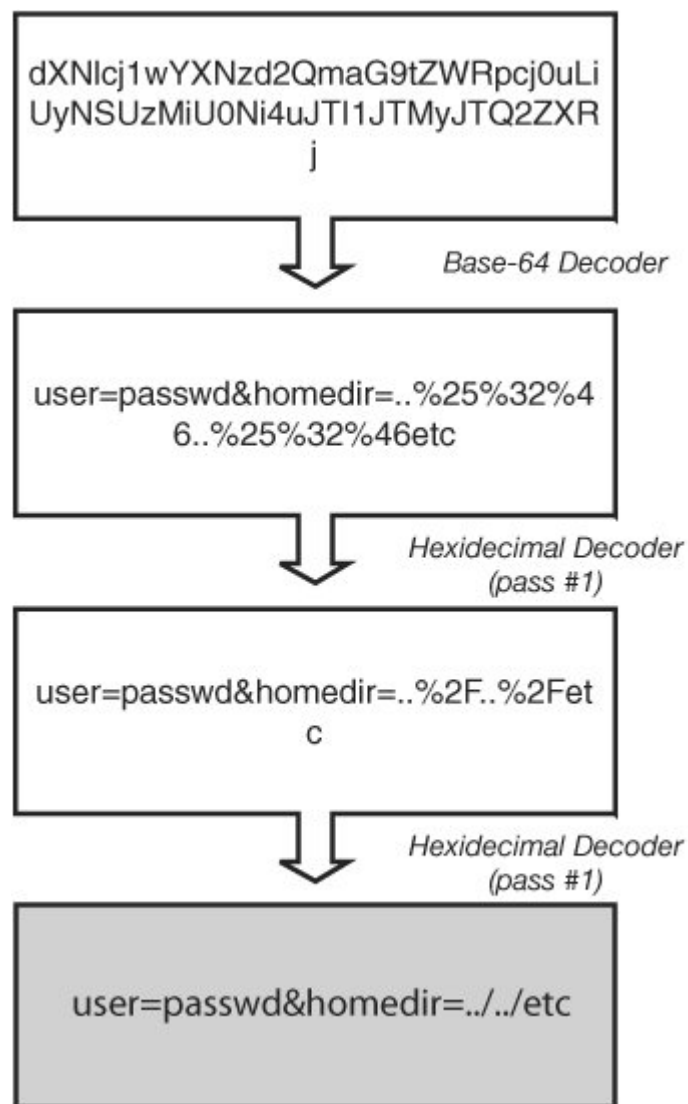
Sometimes data is decoded several times and in several different ways, especially when multiple layers of processing are performed before the input is used for its intended purpose. Decoding several times makes validation extremely difficult, as higher layers see the data in an intermediate format rather than the final unencoded content.

In complex multitiered applications, the fact that input goes through a number of filters or conversions might not be immediately obvious, or it might happen only in certain conditions. For example, data posted to a HTTP Web server might go through base64 decoding if the `Content-Encoding` header specifies this behavior, UTF-8 decoding because it's the encoding format specified in the `Content-Type` header, and finally hexadecimal decoding, which occurs on all HTTP traffic. Additionally, if the data is destined for a Web application or script, it's entirely possible that it goes through another level of hexadecimal decoding. Figure 8-3 shows this behavior.

Figure 8-3. Encoded Web data

This website uses cookies. Click [here](#) to find out more.

Accept cookies



Each component involved in decoding is often developed with no regard to other components performing additional decoding steps at lower or higher layers, so developers might make incorrect judgments on what input should result. Vulnerabilities of this nature tie back into previous discussions on design errors. Specifically, cross-component problems might happen when an interface to a component is known, but the component's exact function is unknown or undefined. For example, a Web server module might perform some decoding of request data to make security decisions about that decoded data. The data might then undergo another layer of decoding afterward, thus introducing the possibility for attackers to sneak encoded content through a filter.

This example brings up another interesting point: Vulnerabilities of this nature might also be a result of operational security flaws. As you learned in Chapter 3, "Operational Review," applications don't operate in a vacuum, especially integrated pieces of software, such as Web applications. The web server and platform modules may provide encoding methods that attackers can use to violate the security of an application.

This website uses cookies. Click [here](#) to find out more.

Accept cookies



# Character Sets and Unicode

[PREVIOUS PAGE](#)[TABLE OF CONTENT](#)[NEXT PAGE](#)

In the previous section, you were primarily concerned with characters that, when left unchecked, might represent a security threat to the application you're reviewing. Extending on this idea, now you examine different character set encodings and common situations in which they can cause problems. Character set encodings determine the sequence of bytes used to represent characters in different languages. In the context of security, you're concerned with how conversions between character sets affects an application's capability to accurately evaluate data streams and filter hostile input.

## Unicode

The Unicode standard describes characters from any language in a unique and unambiguous way. It was primarily intended to address limitations of the ASCII character set and the proliferation of potentially incompatible character sets for other languages. The result is a standard that defines "a consistent way of encoding multilingual text that enables the exchange of text data internationally and creates the foundation for global software." The Unicode standard (available at [www.unicode.org](http://www.unicode.org)) defines characters as a series of codepoints (numerical values) that can be encoded in several formats, each with different size code units. A code unit is a single entity as seen by the encoding and decoding routines; each code unit size can be represented in either byte orderbig endian (BE) or little endian (LE). Table 8-3 shows the different encoding formats in Unicode.

Table 8-3. Unicode Encoding Formats

| Name     | Code Unit Size (in Bits) | Byte Order    |
|----------|--------------------------|---------------|
| UTF-8    | 8                        |               |
| UTF-16BE | 16                       | Big endian    |
| UTF-16LE | 16                       | Little endian |
| UTF-32BE | 32                       | Big endian    |

| Name     | Code Unit Size (in Bits) | Byte Order    |
|----------|--------------------------|---------------|
| UTF-32LE | 32                       | Little endian |

Note that the byte-order specification (BE or LE) can be omitted, in which case a byte-order marker (BOM) at the beginning of the input can be used to indicate the byte order.

These encoding schemes are used extensively in HTTP communications for request data or XML documents. They are also used in a lot of Microsoft-based software because current Windows operating systems use Unicode internally to represent strings. Unicode's codespace is 0 to 0x10FFFF, so 16-bit and 8-bit code units might not be able to represent an entire Unicode character because of size constraints. However, characters can be encoded multibyte streams; that is, several encoded bytes in sequence might combine to represent one Unicode character.

Auditing programs that make use of Unicode characters and Unicode encoding schemes require reviewers to verify:

- o

Whether characters can be encoded to bypass security checks

- o

Whether the implementation of encoding and decoding contains vulnerabilities of its own

The first check requires verifying that characters aren't converted after filter code has run to check the input's integrity. For example, a major bug was discovered in the Microsoft Internet Information Services (IIS) Web server. It was a result of the Web server software failing to decode Unicode escapes before checking whether a user was trying to perform a directory traversal (double dot) attack; so it didn't catch encoded `../` and `..\` sequences. Users could make the following request:

```
GET /..%c0%af..%c0%afwinnt/system32/cmd.exe?/c+dir
```

In this way, they could run arbitrary commands with the permissions the Web server uses.

Note

You can find details of this vulnerability at [www.microsoft.com/security/technet/bulletin/MS00-078.mspx](http://www.microsoft.com/security/technet/bulletin/MS00-078.mspx).

Because many applications use Unicode representation, an attack of this nature is always a major threat. Given that a range of encoding schemes are available to express data, there are quite a few ways to represent the same codepoint. You already know that you can represent a value in 8-, 16-, or 32-bit code units (in either byte order), but smaller code units have multiple ways to represent individual code points. To understand this better, you need to know more about how code points are encoded, explained in the following sections.

#### UTF-8

UTF-8 encoded codepoints are represented as single or multibyte sequences. For the ranges of values 0x00 to 0x7F, only a single byte is required, so the UTF-8 encoding for U.S. ASCII codepoints is identical to ASCII. For other values that can't be represented in 7 bits, a lead byte is given followed by a variable number of trailing bytes (up to four) that combine to represent the value being encoded. The lead byte consists of the highest bit set plus a number of other bits in the most significant word that indicate how many bytes are in this multibyte set. So the number of bits set contiguously in the lead byte's high word specifies the number of trailing bytes, as shown in Table 8-4.

Table 8-4. UTF-8 Lead-Byte Encoding Scheme

| Bit Pattern | Bytes Following |
|-------------|-----------------|
| 110x xxxx   | 1               |
| 1110 xxxx   | 2               |
| 1111 xxxx   | 3, 4, or 5      |

#### Note

The bit pattern rules in Table 8-4 are a slight oversimplification, but they are adequate for the purposes of this discussion. Interested readers are encouraged to browse the current specification at [www.unicode.org](http://www.unicode.org).

The bits replaced by x are used to hold part of the value being represented. Each trailing byte begins with its topmost bits set to 10 and have the least significant 6 bits set to hold part of the value being represented. Therefore, it's illegal for a trailing byte to be less than 0x80 or greater than 0xBF, and it's also illegal for a lead byte to start with 10 (as that would make it indistinguishable from a trailing byte).

Until recently, you could encode Unicode values with any of the supported multibyte lengths you wanted. So, for example, a / character could be represented as

- o

0x2F

- o

0xC0 0xAF

- o

0xE0 0x80 0xAF

- o

0xF0 0x80 0x80 0xAF

The Unicode 3.0 standard, released in 1999, has been revised to allow only the shortest form encoding; for instance, the only legal UTF-8 encoding in the preceding list is 0x2F. Windows XP and later enforce the shortest-form encoding rule. However, not all Unicode implementations are compliant. In particular, ASCII characters are often accepted as one- or two-byte sequences, which could be useful in evading character filters. For example, a filter searching for slashes in a path argument (0x2F) might miss the sequence 0xC0 0xAF; if UTF-8 conversions are performed later, this character filter can be completely evaded for any arbitrary ASCII character.

#### Note

Daniel J. Roelker published an interesting paper on combining these different multibyte encodings with several other hexadecimal encoding techniques to evade IDS filtering of HTTP traffic. It's available at [http://docs.idsresearch.org/http\\_ids\\_evasions.pdf](http://docs.idsresearch.org/http_ids_evasions.pdf).

#### UTF-16

UTF-16 expresses codepoints as 16-bit words, which is enough to represent most commonly used characters in major languages. However, some characters require more than 16 bits. Remember, the codespace for Unicode ranges from 0 to 0x10FFFF, and the maximum value a 16-bit integer can represent is 0xFFFF. Therefore, UTF-16 can also contain multi-unit sequences, so UTF-16 encoded codepoints can be one or two units. A codepoint higher than 0xFFFF requires two code units to be expressed and is encoded as a surrogate pair; that is, a pair of code units with a special lead bit sequence that combines to represent a codepoint. These are the rules for encoding Unicode codepoints in UTF-16 (taken from RFC 2781):

1. If  $U < 0x10000$ , encode  $U$  as a 16-bit unsigned integer and terminate.
2. Let  $U' = U - 0x10000$ . Because  $U$  is less than or equal to 0x10FFFF,  $U'$  must be less than or equal to 0xFFFF. That is,  $U'$  can be represented in 20 bits.

3. Initialize two 16-bit unsigned integers, W1 and W2, to 0xD800 and 0xDC00, respectively. Each integer has 10 bits free to encode the character value, for a total of 20 bits.
4. Assign the 10 high-order bits of the 20-bit U' to the 10 low-order bits of W1 and the 10 low-order bits of U' to the 10 low-order bits of W2. Terminate.

Because the constant value 0x100000 is added to the bits read from a surrogate pair, you can't encode arbitrary values the way you were able to in UTF-8. With UTF-16 encoding, there's only one way to represent a codepoint.

#### UTF-32

UTF-32 expresses codepoints as 32-bit value. Because it can represent any codepoint in a single value, surrogate pairs are never required, as they are in UTF-8 and UTF-16. The only way to alter how a codepoint is represented in UTF-32 encoding is to change the data stream's endian format (using the special BOM mentioned after Table 8-3).

#### Vulnerabilities in Decoding

As mentioned, the difficulty with filtering Unicode data correctly is that the same value can be represented in many ways by using different word-size encodings, by switching byte order, and by abusing UTF-8's unique capability to represent the same value in more than one way. An application isn't going to be susceptible to bypassing filters if only one data decoding is performed—that is, the data is decoded, checked, and then used. However, in the context of HTTP traffic, only one decoding seldom happens. Why? Web applications have increased the complexity of HTTP exchanges dramatically, and data can often be decoded several times and in several ways. For example, the IIS Web server decodes hexadecimal sequences in a request, and then later performs UTF-8 decoding on it and then might hand it off to an ISAPI filter or Web application that does more hexadecimal decoding on it.

#### Note

You can find excellent information on security issues with Unicode in TR36Unicode Security Considerations Technical Report. At the time of this writing, it's available at [www.unicode.org/reports/tr36/](http://www.unicode.org/reports/tr36/).

#### Homographic Attacks

Homographic attacks are primarily useful as a form of social engineering; Evgeniy Gabrilovich and Alex Gontmakher originally described them in "The Homographic Attack" published in the February 2002 edition of Communications of the ACM. These

attacks take advantage of a Unicode homograph, which includes different characters that have the same visual representation. On its simplest level, a homographic attack doesn't specifically require Unicode. For example, the digit 1 (ASCII 0x31) can look like the lowercase letter l (ASCII 0x6c). However, with a little scrutiny, you can tell them apart. In contrast, a Unicode homographic attack involves two graphical representations that are identical, even though the underlying characters are different. For example, the Cyrillic character at codepoint 0x0441 happens to look a lot like the Latin-1 (ASCII) character 0x0063. In general, both are actually rendered as a lowercase c.

Chapter 17 includes an example of a well-publicized homographic attack in the discussion on phishing. For now, just understand that attackers can take advantage of these simple tricks when you're reviewing an application that presents users with data from external sources. Even if the data isn't directly harmful, attackers might be able to use it to trick unsuspecting users.

## Windows Unicode Functions

The Windows operating system deals with string data internally as wide characters (encoded as UTF-16). Because many applications deal with ASCII strings (or perhaps other single or multibyte character sets), Windows provides functions for converting between the two formats as well as ASCII wrapper functions for all the exposed API functions that would otherwise require wide character strings.

The conversion between character encodings takes place similarly whether an application uses ASCII wrapper functions or converts data explicitly. The rules for these conversions are determined primarily by the behavior of two functions: `MultiByteToWideChar()` and `WideCharToMultiByte()`. The details of how these functions perform conversions have a number of security implications ranging from memory corruption errors to conversions that produce unexpected results, as discussed in the following sections.

### `MultiByteToWideChar()`

The `MultiByteToWideChar()` function is used to convert multi- and single-byte character strings into Unicode strings. A maximum of `cchWideChar` characters can be written to the output buffer (`lpWideCharString`). A common error that application developers make when using this function is to specify the destination buffer's size in bytes as the `cchWideChar` parameter. Doing this means twice as many bytes could be written to the output buffer than space has been allocated for, and a buffer overflow might occur. The `MultiByteToWideChar()` function is summarized in the following list:

- o

**Function** `int MultiByteToWideChar(UINT CodePage, DWORD dwFlags, LPCSTR lpMultiByteStr, int cbMultiByte, LPWSTR lpWideCharStr, int cchWideChar)`

- o

API Win32 API

- o

Similar functions `mbtowc`

- o

Purpose `MultiByteToWideChar()` maps a single- or multibyte character string to a wide character string.

The following code is an example misusing `MultiByteToWideChar()`:

```
HANDLE OpenFile(LPSTR lpFilename) { WCHAR wPath[MAX_PATH];
if(MultiByteToWideChar(0, 0, lpFilename, -1, wPath,
sizeof(wPath)) == 0) Return INVALID_HANDLE_VALUE; ...
Create the file ... }
```

This code is an example of the problem just mentioned. The bolded line shows the wide character count is set to the size of the output buffer, which in this case is `MAX_PATH * sizeof(WCHAR)`. However, a `WCHAR` is two bytes, so the output size provided to `MultiByteToWideChar()` is interpreted as `MAX_PATH * 2` bytestwice the real length of the output buffer.

`WideCharToMultiByte()`

The `WideCharToMultiByte()` function is the inverse of `MultiByteToWideChar()`; it converts a string of wide characters into a string of narrow characters. Developers are considerably less likely to trigger a buffer overflow when using this function because the output size is in bytes rather than wide characters, so there's no misunderstanding the meaning of the size parameter. The `WideCharToMultiByte()` function is summarized in the following list:

- o

Function `int WideCharToMultiByte(UINT CodePage, DWORD dwFlags, LPCWSTR lpWideCharStr, int cchWideChar, LPSTR lpMultiByteStr, int cbMultiByte, LPCSTR lpDefaultChar, LPBOOL lpUsedDefaultChar)`

- o

API Win32 API

- o

Similar functions `wctombc`

- o

Purpose `WideCharToMultiByte()` maps a wide character string to a single- or multibyte character string.

Because wide characters are a larger data type, their information sometimes needs to be represented by a sequence of single-bytes, called a multibyte character. The rules for encoding wide characters into multibyte characters are governed by the code page specified as the first argument to this function.

#### NUL-Termination Problems

The `MultiByteToWideChar()` and `WideCharToMultiByte()` functions don't guarantee NUL-termination if the destination buffer is filled. In these cases, the functions return 0, as opposed to the number of characters converted. It's intended that users of these functions check the return value; however, this is often not the case. Listing 8-29 shows a brief example.

Listing 8-29. Return Value Checking of `MultiByteToWideChar()`

```
HANDLE open_file(char *name) { WCHAR buf[1024]; HANDLE
hFile; MultiByteToWideChar(CP_ACP, 0, name,
strlen(filename), buf, sizeof(buf)/2);
wcsncat(buf, sizeof(buf)/2 wcslen(buf) 1, ".txt"); ... }
```

Because the return value is left unchecked, the fact that `buf` isn't big enough to hold the name being converted isn't caught, and `buf` is not NUL-terminated. This causes `wcsncat()` to miscalculate the remaining buffer size as a negative number, which you know is converted into a large positive number if you review the `wcsncat()` function prototype listed under `strncat()`.

`MultiByteToWideChar()` might have additional problems when multibyte character sets are being converted. If the `MB_ERR_INVALID_CHARS` flag is specified, the function triggers an error when an invalid multibyte sequence is encountered. Here's an example showing a potentially dangerous call:

```
PWCHAR convert_string(UINT cp, char *instr) { WCHAR *outstr;
size_t length; length = strlen(instr) + 1; outstr = (WCHAR
*)calloc(length, sizeof(WCHAR)); MultiByteToWideChar(cp,
MB_ERR_INVALID_CHARS, instr, -1, outstr, -1);
return outstr; }
```

Again, because the function's return value isn't checked, the `convert_string()` function doesn't catch invalid character sequences. The problem is that `MultiByteToWideChar()` returns an error when it sees an invalid character sequence, but it doesn't NUL-terminate the destination buffer (`outstr`, in this case). Because the return value isn't checked, the function doesn't deal with this error, and an unterminated wide string is

returned to the caller. Because of this any later processing on this string could result in memory corruption.

#### Unicode Manipulation Vulnerabilities

Memory management issues can also occur when using any bounded multibyte or wide character functions. Take a look at an example using `wcsncpy()`:

```
wchar_t destination[1024]; wcsncpy(destination, source,
sizeof(destination));
```

At first glance, it seems as though this code is correct, but of course the size parameter should indicate how big the destination buffer is in wide characters, not the size in bytes; so the third argument is actually twice the length of the output buffer. This mistake is easy to make, so code auditors should keep an eye out for it.

Another interesting quirk is errors in dealing with user-supplied multibyte-character data strings. If the application code page indicates that a double-byte character set (DBCS) is in use, characters can be one or two bytes. Applications processing these strings need to identify whether each byte being processed is a single character or part of a two-byte sequence; in Windows, this check is performed with the `IsDBCSLeadByte()` function. Vulnerabilities in which a pointer can be incremented out of the buffer's bounds can easily occur if the application determines that a byte is the first of a two-byte sequence and neglects to check the next byte to make sure it isn't the terminating NUL byte. Listing 8-30 shows an example.

Listing 8-30. Dangerous Use of `IsDBCSLeadByte()`

```
char *escape_string(char *src) { char *newstring, *dst;
newstring = (char *)malloc(2*strlen(src) + 1);
if(!newstring) return NULL; for(dst = newstring;
*src; src++){ if(IsDBCSLeadByte(*src)){
*dsrc++ = *src++; *dst++ = *src;
continue; } if(*src == '\\'))
*dsrc++ = '\\'; *dst++ = *src; } return
newstring; }
```

When the code in Listing 8-30 encounters a lead byte of a two-byte sequence, it does no checking on the second byte of the two-byte sequence. If the string passed to this function ends with a DBCS lead byte, the lead byte and the terminating NUL byte are written to the destination buffer. The `src` pointer is incremented past the NUL byte and continues processing bytes outside the bounds of the string passed to this function.

This error could result in a buffer overflow of the `newstring` buffer, as the allocated length is based on the string length of the source string.

#### Note

When multibyte character sequences are interpreted, examine the code to see what can happen if the second byte of the sequence is the string's terminating NUL byte. If no check is done on the second byte, processing data outside the buffer's bounds might be possible.

#### Code Page Assumptions

When converting from multibyte to wide characters, the code page argument affects how `MultiByteToWideChar()` behaves, as it specifies the character set the multibyte string is encoded in. In most cases, this function is used with the default system code page (`CP_ACP`, ANSI Code Page), which doesn't do much. However, attackers can affect the code page in some situations by constructing multibyte character sequences that evade filters in earlier layers. Listing 8-31 is an example of a vulnerable code fragment.

Listing 8-31. Code Page Mismatch Example

```
if(strchr(filename, '/') || strchr(filename, '\\')){
 error("filenames with slashes are illegal!"); return 1; }
MultiByteToWideChar(CP_UTF8, 0, filename, strlen(filename),
wfilename, sizeof(wfilename)/2); ...
```

As you can see, encoding is performed after a check for slashes, so by encoding slashes, attackers targeting earlier versions of Windows can evade that check and presumably do something they shouldn't be able to later. Akio Ishida and Yasuo Ohgaki discovered an interesting variation on this vulnerability in the PostgreSQL and MySQL database APIs (available at [www.postgresql.org/docs/techdocs.50](http://www.postgresql.org/docs/techdocs.50)). As mentioned, SQL control characters are commonly escaped with the backslash (`\`) character. However, some naive implementations of this technique might not account for multibyte characters correctly. Consider the following sequence of bytes:

`0x95 0x5c 0x27`

It's actually a string in which the first two bytes are a valid Shift-JIS encoded Japanese character, and the last byte is an ASCII single quote (`'`). A naive filter won't identify that the first two bytes refer to one character; instead, it interprets the `0x5c` byte as the backslash character. Escaping this sequence would result in the following bytes:

0x95 0x5c 0x5c 0x5c 0x27

Passing the resulting string to a multibyte-aware database can cause a problem because the first two bytes are interpreted as a single Japanese character. Then the remaining two 0x5c bytes are interpreted as an escaped backslash sequence. Finally, the last byte is left as an unescaped single quote character. This misinterpreted encoding can be exploited to inject SQL statements into an application that otherwise shouldn't be vulnerable.

Having multibyte character sets used with `MultiByteToWideChar()` might have some additional complications related to memory corruption. Listing 8-32 shows an interesting call to this function.

Listing 8-32. NUL Bytes in Multibyte Code Pages

```
PWCHAR convert_string(UINT cp, char *instr) { WCHAR
*outstr; size_t length; length = strlen(instr) * 2 +
1; outstr = (WCHAR *)calloc(length, sizeof(WCHAR));
MultiByteToWideChar(cp, 0, instr, -1, outstr, -1); return
outstr; }
```

The `MultiByteToWideChar()` function in Listing 8-32 is vulnerable to a buffer overflow when a multibyte code page is used. Why? Because the output string length is calculated by multiplying the input string length by two. However, this calculation isn't adequate because the NUL byte in the string could be part of a multibyte character; therefore, the NUL byte can be skipped and out-of-bounds characters continue to be processed and written to the output buffer. In UTF-8, if the NUL byte appeared in a multibyte sequence, it would form an illegal character; however, `MultiByteToWideChar()` enters a default replacement or skips the character (depending on Windows versions), unless the `MB_ERR_INVALID_CHARS` flag is specified in the second argument. When that flag is specified, the function returns an error when it encounters an illegal character sequence.

#### Character Equivalence

Using `WideCharToMultiByte()` has some interesting consequences when decoding data. If conversions are performed after character filters, the code is equally susceptible to sneaking illegal characters through filters. When converting wide characters into multibyte, however, the risk increases for two main reasons:

- o

Even with the default code page, multiple 16-bit values often map to the same 8-bit character. As an example, if you want a backslash to appear in the input stream of the

converted character set, you can supply three different wide characters that convert into the backslash byte (0x5c): 0x00 0x5c, 0x22 0x16, and 0xff 0x0c. You can do this not because the backslash character has three Unicode representations, but because output character represents the closest match when an exact conversion can't be performed. This behavior can be toggled with the `WC_NO_BEST_FIT_CHARS` flag.

- o

When a character is encountered that can't be converted to a multibyte character and a close replacement can't be found (or the `WC_NO_BEST_FIT` flag is set), a default replacement character is inserted in the output stream; the `.` character is used for the ANSI code page, unless otherwise specified. If this replacement character is filtered, a wide range of values can generate this character in the output stream.

Auditing code that uses `MultiByteToWideChar()` or `WideCharToMultiByte()` requires careful attention to all these factors:

- o

Check whether data is required to pass through a filter before it's converted rather than after.

- o

Check whether the code page is multibyte or can be specified by a user.

- o

If the `MB_ERR_INVALID_CHARS` flag is set for converting multibyte streams, user input can prematurely terminate processing and leave an unterminated output buffer. If it's omitted, a multibyte sequence including the trailing NUL byte can be specified, potentially causing problems for poorly written code.

- o

If the `WC_NO_BEST_FIT_CHARS` flag is present for converting wide character data, users might be able to supply multiple data values that translate into the same single-byte character. The best-fit matching rules are years out of date, and most developers shouldn't use them anyway.

- o

Look for any other flags affecting how these functions might be misused.

- o

Make sure the return value is checked. If a conversion error is not identified, unterminated buffers might be used incorrectly.

- o

Check the sizes of input and output buffers, as mentioned in the discussion of memory corruption in Chapter 5.